# Lab Assignment 6 Cracking Passwords

## Cracking Results and Rules

shimate_1

Password: regina

Default john single/rules and the shipped wordlist. The commands used:

/snap/bin/john --format=md5crypt-long shimate_1
/snap/bin/john --show shimate_1

```
exouser@shimate:~$ /snap/bin/john --format=md5crypt-long shimate_1
Created directory: /home/exouser/snap/john-the-ripper/694/.john
Created directory: /home/exouser/snap/john-the-ripper/694/.john/opencl
Using default input encoding: UTF-8
Loaded 1 password hash (md5crypt-long, crypt(3) $1$ (and variants) [MD5 32/
64])
Will run 8 OpenMP threads
Proceeding with single, rules:Single
Press 'q' or Ctrl-C to abort, 'h' for help, almost any other key for status
Almost done: Processing the remaining buffered candidate passwords, if any.
0g 0:00:00:00 DONE 1/3 (2025-11-01 22:54) 0g/s 10180p/s 10180c/s 10180C/s S
himate1905..Shimate1900
Proceeding with wordlist:/snap/john-the-ripper/current/bin/password.lst
Enabling duplicate candidate password suppressor
regina           (shimate)
1g 0:00:00:00 DONE 2/3 (2025-11-01 22:54) 3.448g/s 11896p/s 11896c/s 11896C
/s cheerleader..0987654321
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

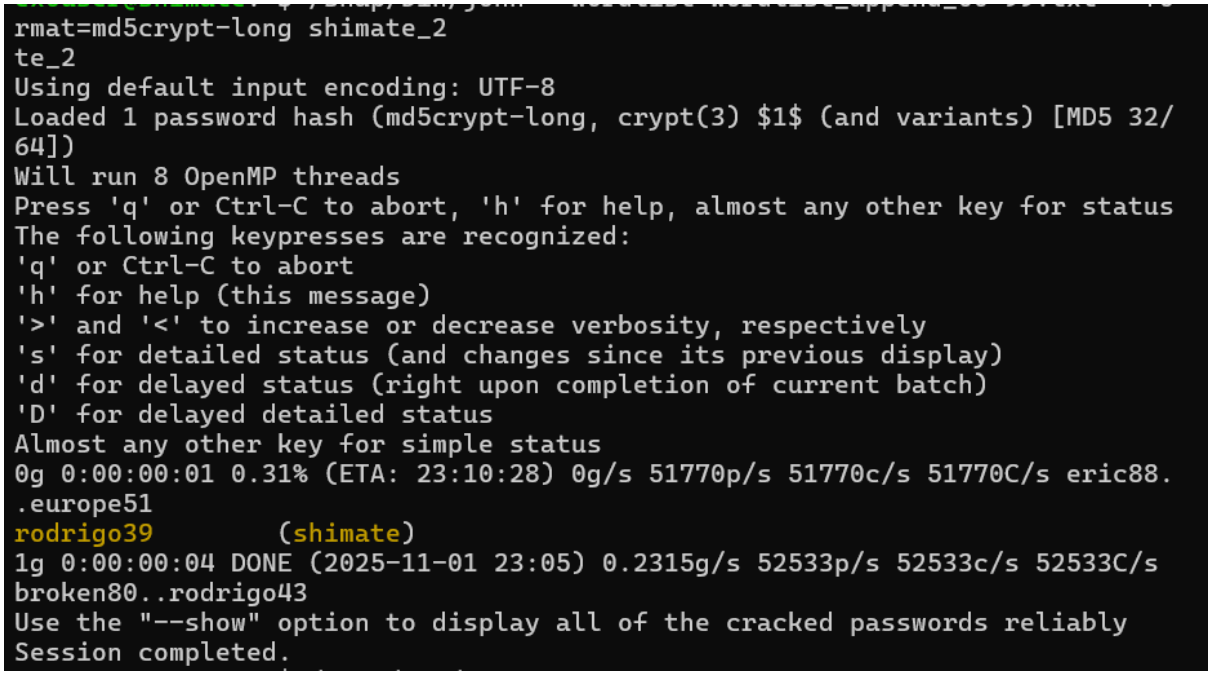
## shimate_2

Password: rodrigo39

Created a local john.conf with the Wordlist rule that appends two digits ($[0-9]$[0-9]) and ran John with --rules=Wordlist against the shipped wordlist:

```
cat > ~/john.conf <<'EOF'
[List.Rules:Wordlist]
$[0-9]$[0-9]
EOF
```

```
/snap/bin/john --config=~/john.conf --wordlist=/snap/john-the-ripper/current/bin/password.lst --rules=Wordlist --format=md5crypt-long shimate_2
/snap/bin/john --show shimate_2
```



## shimate_3

Password: su22sana11

Generated candidate transformations that insert two digits after the 2nd character and append two digits and streamed candidates into John by building a john.conf ruleset. Commands and the rules used are as follows:

cat > ~/john.conf <<'EOF'
[List.Rules:Wordlist]
A2"11"Az"11"
A2"11"Az"12"
A2"11"Az"21"
A2"11"Az"22"
A2"12"Az"11"
A2"12"Az"12"
A2"12"Az"21"
A2"12"Az"22"
A2"21"Az"11"
A2"21"Az"12"
A2"21"Az"21"
A2"21"Az"22"
A2"22"Az"11"
A2"22"Az"12"
A2"22"Az"21"
A2"22"Az"22"
EOF

/snap/bin/john --config=~/john.conf --wordlist=/snap/john-the-ripper/current/bin/password.lst --rules=Wordlist --format=md5crypt-long shimate_3
/snap/bin/john --show shimate_3

```
exouser@shimate:~$ /snap/bin/john --config=~/john.conf --wordlist=/snap/joh
n-the-ripper/current/bin/password.lst --rules=Wordlist --format=md5crypt-lo
ng shimate_3
Loaded 1 password hash (md5crypt-long, crypt(3) $1$ (and variants) [MD5 32/
64])
Will run 8 OpenMP threads
Press 'q' or Ctrl-C to abort, 'h' for help, almost any other key for status
Enabling duplicate candidate password suppressor
The following keypresses are recognized:
'q' or Ctrl-C to abort
'h' for help (this message)
'>' and '<' to increase or decrease verbosity, respectively
's' for detailed status (and changes since its previous display)
'd' for delayed status (right upon completion of current batch)
'D' for delayed detailed status
Almost any other key for simple status
su22sana11       (shimate)
1g 0:00:06:46 DONE (2025-11-01 23:58) 0.002463g/s 53077p/s 53077c/s 53077C/
s si22ster11..il22ovejesus11
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

## shimate_4

Password: noors

A Dutch wordlist was downloaded and ran John with that list:

wget https://download.openwall.net/pub/wordlists/languages/Dutch/1-clean/lower.gz

gunzip lower.gz

mv lower dutch

/snap/bin/john --wordlist=./dutch --format=md5crypt-long shimate_4

/snap/bin/john --show shimate_4

```
exouser@shimate:~$ /snap/bin/john --wordlist=/home/exouser/dutch --format=m
d5crypt-long shimate_4
Using default input encoding: UTF-8
Loaded 1 password hash (md5crypt-long, crypt(3) $1$ (and variants) [MD5 32/
64])
Will run 8 OpenMP threads
Press 'q' or Ctrl-C to abort, 'h' for help, almost any other key for status
noors            (shimate)
1g 0:00:00:02 DONE (2025-11-02 00:00) 0.4695g/s 48045p/s 48045c/s 48045C/s
noordoostelijker..noppig
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
exouser@shimate:~$
```

The 4 passwords were verified using the --show command:

```
exouser@shimate:~$ /snap/bin/john --show shimate_1 shimate_2 shimate_3 shim
ate_4
shimate:regina
shimate:rodrigo39
shimate:su22sana11
shimate:noors

4 password hashes cracked, 0 left
```

# Questions and answers

1. Explain how a “white hat” security professional might use John-the-Ripper to make her institution more secure

A white-hat security professional uses John the Ripper to perform proactive password auditing. By running John against exported, hashed credential files in a controlled environment, the professional can discover weak or commonly used passwords and demonstrate which accounts are at risk. The results guide concrete remediation like enforcing stronger password policies, rotating compromised credentials, requiring multi-factor authentication for exposed accounts, and educating users about secure password creation. This helps the institution reduce the attack surface before an adversary exploits weak credentials.

2. Why can John-the-Ripper crack passwords even though they are not in a form that is directly readable?

Passwords are typically stored as salted or unsalted hashes rather than plaintext, but hashing is deterministic like the same password always yields the same hash under the same algorithm and salt. John the Ripper generates candidate plaintexts, computes their hashes using the same hash function and salt, and compares those results to the stored hashes. When a computed hash matches a stored hash, John has found the original password. In short, John succeeds by reproducing the hash computation for many plausible candidate passwords and checking for matches.

3. Explain how John-the-Ripper limits/reduces password cracking time?

John reduces cracking time with several strategies like it applies intelligent wordlists, so likely passwords are tried first then it uses rule-based mangling like appending digits, capitalizing letters, to produce many plausible variants from a smaller base dictionary then it supports hybrid attacks that combine dictionary words with brute force suffixes/prefixes and then it leverages parallelism to increase candidates-per-second. John also supports streaming candidates, candidate suppression for duplicates, and incremental modes that focus on character sets and lengths, all designed to prioritize high probability guesses and reduce overall work.

4. Why system policies for passwords would make user passwords considerably harder to crack by a password cracker such as John-the-Ripper? What are the downsides of forcing such policies?

Stronger policies that significantly increase cracking difficulty, include enforcing long minimum lengths, requiring passphrases or mixed character sets, banning use of common words and predictable patterns, and enforcing per-user unique salts combined with slow hashing algorithms. These measures raise the entropy and computational cost for each guess, forcing attackers to spend far more time per candidate. Downsides include user friction i.e. difficulty remembering complex passwords, increased help-desk workload i.e. more password resets, potential insecure user behaviours i.e. writing passwords down or reuse across services, and higher server cost/performance impact when using slow hash functions for authentication operations. A careful policy balances security like encouraging passphrases, enforcing MFA with usability to avoid counterproductive workarounds.

5. How much does a salt of size N increases processing required by precomputed dictionary offline attacks?

A salt prevents simple reuse of precomputed hash tables by forcing a unique hash output per distinct salt. If an attacker wants to precompute hashes for every salt value, salts of size N bits increase the precomputation space by a factor of 2^N i.e. an attacker would need to compute and store 2^N times more hashes to cover all salt values. In practice, salts are typically long enough like 64 bits or more that precomputing across the full salt space becomes infeasible, so salts shift attackers to brute-force or online offline attacks per account rather than one large shared precomputation.